

INDEX

Sr. No.	Chapter Name	Page No.
1	INTRODUCTION	1
2	PROBLEM STATEMENT	2
3	OBJECTIVES	3
4	SYSTEM REQUIREMENT	4
5	SCOPE OF THE PROJECT	5
6	LITERATURE SURVEY	6
7	SYSTEM ANALYSIS	7
8	PROPOSED METHODOLOGY	8
9	DATASET DESCRIPTION	10
10	EXPLORATORY DATA ANALYSIS	12
11	DATA VISUALIZATION	16
12	FEATURE SCALING	23
13	TRAIN-TEST SPLIT	25
14	DATA IMBALANCE HANDLING	28
15	MODEL BUILDING	31
16	CONCLUSION AND FUTURE BENEFITS	35
17	REFERENCES	37

1. INTRODUCTION

Nowadays, credit cards have become an important part of our daily life. People use them for online shopping, bill payments, and many other digital transactions because they are fast, easy, and convenient. As the use of credit cards continues to grow, the number of fraudulent transactions is also increasing. Credit card fraud has become a serious issue because it can cause financial loss to both customers and banks.

Credit card fraud happens when someone uses another person's credit card or card information without the owner's permission. In many cases, the cardholder is not aware that their card details have been stolen or misused. Therefore, it is important to have a system that can quickly detect suspicious transactions and prevent fraud before it causes major damage.

Machine Learning is one of the most effective technologies for detecting credit card fraud. It can analyse a large number of transactions, learn different transaction patterns, and identify unusual activities. By using Machine Learning, banks and financial institutions can detect fraudulent transactions more accurately and take immediate action.

In this project, a **Credit Card Fraud Detection** System is developed using Machine Learning techniques. The transaction dataset is collected and then prepared for analysis by handling missing values, removing duplicate records, and performing the necessary data preprocessing steps. After preprocessing, a Machine Learning model is trained to classify each transaction as genuine or fraudulent. The model is then tested and evaluated using different performance metrics.



The main aim of this project is to improve the accuracy of fraud detection and reduce financial losses. This system can help banks and financial institutions provide safer online transactions, protect customer information, and increase trust in digital payment systems.

2. PROBLEM STATEMENT

Credit cards are used every day for shopping, bill payments, and many other online transactions. As the number of digital payments increases, the number of credit card fraud cases is also increasing. This creates a serious problem for banks as well as customers.

It is not easy to identify whether a transaction is genuine or fraudulent. Fraudsters keep changing their methods, so traditional fraud detection systems cannot always detect every fraud. Sometimes, genuine transactions are blocked, while fraudulent transactions are approved, which can lead to financial loss.

Another challenge is that the transaction dataset contains a very small number of fraudulent transactions compared to genuine ones. Because of this imbalance, it becomes difficult to train a Machine Learning model that can accurately detect fraud.

To overcome these problems, there is a need for a reliable fraud detection system. The system should analyse transaction data, identify suspicious activities, and detect fraudulent transactions quickly and accurately. This can help reduce financial losses and improve the security of online payment systems.

3. OBJECTIVES

The main objective of this project is to develop a Credit Card Fraud Detection System using Machine Learning. The project focuses on identifying fraudulent transactions and improving the security of online payment systems.

Another objective is to analyse the credit card transaction dataset and prepare it for model training. This includes data preprocessing tasks such as handling missing values, removing duplicate records, and preparing the data for better analysis.

The project also aims to build a Machine Learning model that can accurately classify transactions as genuine or fraudulent. Finally, the model is evaluated using suitable performance metrics to improve fraud detection, reduce financial losses, and provide a more secure payment experience for customers.

4. SYSTEM REQUIREMENT

The Credit Card Fraud Detection project requires some basic software and hardware to develop and run the Machine Learning model. The system requirements are divided into two parts: software requirements and hardware requirements.

2.1 Software Requirement

The following software and tools are used for developing the Credit Card Fraud Detection project:

- **Operating System:** Windows 10 / Windows 11
- **Programming Language:** Python
- **Development Environment:** Jupyter Notebook
- **Libraries:** Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
- **Data Processing:** Pandas and NumPy
- **Data Visualization:** Matplotlib and Seaborn
- **Machine Learning:** Scikit-learn
- **Dataset Format:** CSV
- **Code Management:** Git and GitHub

These software tools are used for data cleaning, data analysis, visualization, model building, and performance evaluation.

2.2 Hardware Requirement

The project does not require high-end hardware because the Machine Learning model is trained on a normal computer. The basic hardware requirements are:

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 4 GB, 8 GB recommended
- **Storage:** Minimum 10 GB free space
- **System Type:** 64-bit
- **Internet Connection:** Required for downloading datasets, libraries, and using GitHub
- **Keyboard and Mouse:** Standard keyboard and mouse

5. SCOPE OF THE PROJECT

The scope of this project is to develop a Machine Learning based system that can help in detecting fraudulent credit card transactions. The system analyses transaction details and identifies whether a transaction is genuine or fraudulent.

This project can be useful for banks, financial institutions, and online payment services. It can help them identify suspicious transactions at an early stage and reduce the chances of financial loss. The system can also help in improving the security of digital payment services and protecting customers from unauthorized transactions.

The project can be further improved by using larger and more recent transaction datasets. Different Machine Learning algorithms can also be tested to improve the prediction results. In the future, the system can be connected with a real-time payment system so that suspicious transactions can be detected immediately.

6. LITERATURE SURVEY

Credit card fraud detection has been studied by many researchers because fraud is a common problem in the financial sector. Different Machine Learning techniques have been used to identify fraudulent transactions from large amounts of transaction data.

Some studies have used **Logistic Regression** for fraud detection because it is simple and useful for classification problems. It can classify a transaction into two classes, such as genuine and fraudulent. Other studies have used algorithms such as **Decision Tree, Random Forest, K-Nearest Neighbors (KNN)** to improve fraud detection results.

Researchers have also focused on the problem of **imbalanced data** in credit card fraud detection. In most datasets, genuine transactions are much higher than fraudulent transactions. Because of this, accuracy alone is not enough to judge the performance of a model. Measures such as precision, recall, F1-score, and confusion matrix are also useful for evaluating the model.

From the previous studies, it can be understood that Machine Learning can be useful for detecting credit card fraud. Different algorithms give different results depending on the dataset and preprocessing methods used. Based on this understanding, this project uses Machine Learning techniques to analyse credit card transactions and identify fraudulent transactions.

7. SYSTEM ANALYSIS

System analysis is the process of understanding the existing problem and deciding how the proposed system can solve it. In this project, the analysis is mainly focused on the problems faced while detecting fraudulent credit card transactions.

Existing System

In the existing system, credit card fraud is detected using different rules, manual checking, and automated fraud detection methods. These methods check transaction details and try to find unusual activities. However, it can be difficult to detect all types of fraud because fraudsters keep changing their methods. Another problem is that a large number of transactions are processed every day. Checking all transactions manually is time-consuming and may also lead to errors. Some genuine transactions may be considered fraudulent, while some actual fraud transactions may not be detected.

Proposed System

The proposed system uses Machine Learning to detect fraudulent credit card transactions. The transaction data is first cleaned and prepared for analysis. After that, the required features are selected and used to train the Machine Learning model.

The trained model learns from previous transaction data and predicts whether a new transaction is genuine or fraudulent. The model performance is then checked using different evaluation metrics.

This approach can make the fraud detection process faster and more systematic. It can also help in reducing manual effort and identifying suspicious transactions more effectively.

Advantages of Proposed System

The proposed system provides several benefits. It can process a large amount of transaction data and identify suspicious transactions in less time. Machine Learning also helps the system learn patterns from previous transactions. This can reduce manual checking and support banks in taking faster action against fraudulent activities.

The system can also be improved in the future by using more data and testing different Machine Learning algorithms. This makes it suitable for further development and real-world fraud detection applications.

8. PROPOSED METHODOLOGY

The proposed methodology explains the steps followed to develop the Credit Card Fraud Detection system. The main purpose of this methodology is to prepare the transaction data, train a Machine Learning model, and predict whether a transaction is genuine or fraudulent.

Data Collection

The first step of the project is to collect the credit card transaction dataset. The dataset contains different details related to transactions, such as transaction amount, transaction category, customer information, location details, and fraud status. The dataset is stored in CSV format and is used for further analysis.

Data Preprocessing

After collecting the dataset, the data is checked and cleaned before using it for Machine Learning. Missing values and duplicate records are checked and handled. Unnecessary columns are removed, and categorical data is converted into a suitable format. The required features are then prepared for model training.

Exploratory Data Analysis

Exploratory Data Analysis is performed to understand the dataset and find useful patterns. Different graphs and statistical methods are used to study the data. Count plots, histograms, box plots, and correlation analysis are used to understand the transaction data and identify important features.

Feature Selection and Scaling

The important features required for fraud detection are selected from the dataset. Numerical features are scaled when required so that the Machine Learning model can process the values properly. This step helps to improve the performance of the model.

Train-Test Split

The prepared dataset is divided into two parts: training data and testing data. The training data is used to teach the Machine Learning model, while the testing data is used to check how well the model performs on unseen data.

Model Building

After preparing the data, a Machine Learning classification model is trained using the training dataset. In this project, **Logistic Regression** is used to classify the transactions into two categories: genuine and fraudulent.

Model Evaluation

After training the model, its performance is checked using the testing dataset. Evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix are used to understand the performance of the model.

Fraud Prediction

Finally, the trained model is used to predict new transactions. Based on the prediction, the transaction is classified as either genuine or fraudulent. This helps in identifying suspicious transactions and supports the fraud detection process.

9. DATASET DESCRIPTION

The dataset used in this project is a credit card transaction dataset. It contains information about different transactions and their related details. The dataset is used to study transaction patterns and identify whether a transaction is genuine or fraudulent.

The dataset contains different types of information such as transaction date and time, transaction amount, merchant details, customer information, location details, and fraud status. These features help in understanding the behaviour of different transactions.

The main target column used in this project is **is_fraud**. It indicates whether a transaction is fraudulent or not. A value of **0** represents a genuine transaction, while a value of **1** represents a fraudulent transaction.

8.1 Dataset Features

Feature/Columns	Description
trans_date_trans_time	Date and time when the transaction took place
cc_num	Credit card number
merchant	Name of the merchant
category	Category of the transaction
amt	Amount of the transaction
first	First name of the cardholder
last	Last name of the cardholder
gender	Gender of the cardholder
street	Street information
city	City of the transaction
state	State of the transaction
zip	ZIP code of the location
lat	Latitude of the transaction location
long	Longitude of the transaction location
city_pop	Population of the transaction city
job	Job of the cardholder
dob	Date of birth of the cardholder
trans_num	Unique transaction number
unix_time	Transaction time in Unix format

Feature/Columns	Description
merch_lat	Latitude of the merchant location
merch_long	Longitude of the merchant location
is_fraud	Shows whether the transaction is genuine or fraudulent

The dataset contains both numerical and categorical features. Numerical features include transaction amount, latitude, longitude, and city population, while categorical features include merchant, category, gender, city, state, and job. These features are analysed and prepared during the data preprocessing stage before applying the Machine Learning model.

10. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is performed to understand the credit card transaction dataset before applying the Machine Learning model. In this project, the dataset contains different details related to transactions, customers, merchants, transaction locations, and fraud status.

The EDA process helps to understand the structure of the data and find important patterns in the transactions. It also helps to identify problems such as missing values, duplicate records, incorrect data types, and unusual values. The analysis is performed using Python libraries such as Pandas and NumPy.

The following steps were performed during the Exploratory Data Analysis.

9.1 Understanding the Dataset

The first step in EDA was to understand the basic structure of the dataset. The dataset contains information about credit card transactions along with customer, merchant, and location details.

The main columns available in the dataset include trans_date_trans_time, cc_num, merchant, category, amt, first, last, gender, street, city, state, zip, lat, long, city_pop, job, dob, trans_num, unix_time, merch_lat, merch_long, and is_fraud.

The trans_date_trans_time column contains the date and time of the transaction, while amt represents the transaction amount. The merchant and category columns provide information about where and what type of transaction was made. Customer-related information is available through columns such as first, last, gender, job, and dob.

The location of the transaction is represented using city, state, zip, lat, and long. The merchant location is given by merch_lat and merch_long. The trans_num column represents the transaction number.

The most important column for this project is is_fraud. This column is used as the target variable. It indicates whether a transaction is genuine or fraudulent.

9.2 Statistical Analysis

After understanding the dataset structure, statistical analysis was performed on the numerical features. This analysis helps to understand the general behaviour and range of the numerical data.

Features such as amt, city_pop, lat, long, merch_lat, and merch_long were considered for numerical analysis. Basic statistical values such as count, mean, standard deviation, minimum value, maximum value, and quartiles were checked.

The amt column is especially important because it represents the amount involved in each transaction. By studying its statistical values, it is possible to

understand the normal range of transaction amounts and identify unusually high or low values.

The statistical analysis also gives an overall idea about the distribution of the numerical features. This information is useful before applying preprocessing and Machine Learning techniques.

9.3 Missing Value Analysis

Missing value analysis was performed to check whether any columns contained null or missing values. Missing values can create problems during data analysis and model training, so they need to be identified before building the model.

The dataset was checked column by column to find the number of missing values. The result helps to decide whether any missing values need to be removed or replaced.

This step is important because the Machine Learning model requires properly prepared data. After checking the missing values, the required preprocessing was performed before continuing with the next steps of the project.

9.4 Duplicate Data Analysis

Duplicate data was also checked during the EDA process. A duplicate record means that the same transaction information is repeated more than once in the dataset.

Duplicate records can affect the analysis because they may give more importance to the same transaction. They can also affect the training and evaluation of the Machine Learning model.

Therefore, the dataset was checked for duplicate rows. If duplicate records were present, they were identified and handled during the data cleaning process. This helped to maintain better quality and consistency in the dataset.

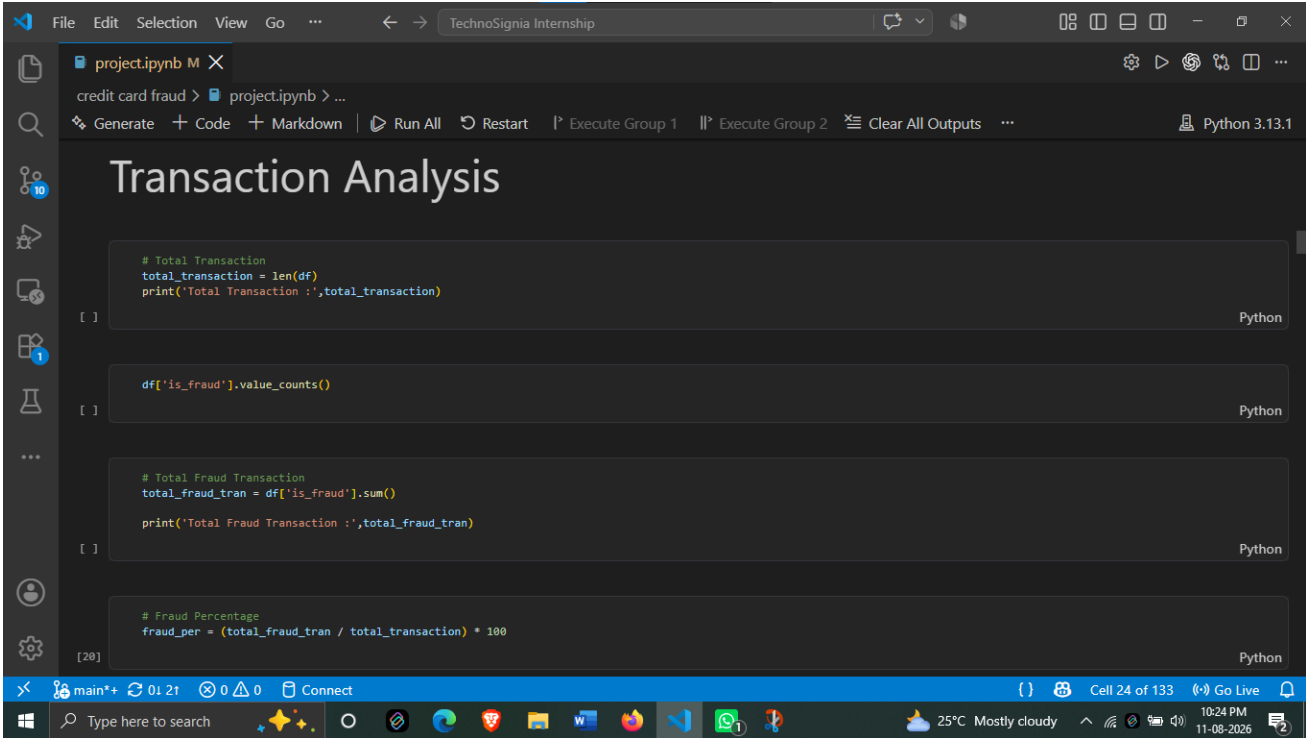
9.5 Transaction Analysis

Transaction analysis was performed to understand the overall behaviour of the transactions in the dataset. The `is_fraud` column was mainly used for this analysis because it separates genuine transactions from fraudulent transactions.

The value 0 in the `is_fraud` column represents a genuine transaction, while the value 1 represents a fraudulent transaction. By comparing these two classes, the distribution of genuine and fraudulent transactions can be understood.

One important observation from credit card fraud datasets is that genuine transactions are usually much higher than fraudulent transactions. This creates an imbalanced dataset. Such imbalance is important to consider while training and evaluating the Machine Learning model because a model can show high accuracy even if it does not detect enough fraudulent transactions.

The transaction information was also studied using features such as merchant, category, city, and state. These features provide information about where and how transactions are being made and can be useful for identifying transaction patterns.



```
# Total Transaction
total_transaction = len(df)
print('Total Transaction :',total_transaction)

df['is_fraud'].value_counts()

# Total Fraud Transaction
total_fraud_tran = df['is_fraud'].sum()
print('Total Fraud Transaction :',total_fraud_tran)

# Fraud Percentage
fraud_per = (total_fraud_tran / total_transaction) * 100
```

9.6 Time Analysis

Time analysis was performed using the `trans_date_trans_time` column. This column contains the date and time at which each transaction took place.

The transaction date and time can provide useful information about customer transaction behaviour. For example, transactions can be grouped according to hour, day, or date to understand when most transactions take place.

Time-related information can also be useful for fraud detection because unusual transaction timings may sometimes indicate suspicious activity. For this reason, the transaction date and time were considered during the analysis and can be further converted into useful features for Machine Learning.

The `unix_time` column also contains the transaction time in Unix timestamp format. This column provides another numerical representation of the transaction time.

```
df['hour'] = df['trans_date_trans_time'].dt.hour

# fraud by Hour
fraud_by_hour = df[df['is_fraud']== 1]['hour'].value_counts().sort_index()
print(fraud_by_hour)

# Fraud by Time Distribution
def time_period(hour):
    if 6 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 18:
        return 'Afternoon'
    elif 18 <= hour < 24:
        return 'Evening'
    else:
        return 'Night'
```

9.7 Amount Analysis

The amt column represents the amount of each credit card transaction. Amount analysis was performed to understand how transaction values are distributed in the dataset.

The minimum, maximum, average, and other statistical values of the transaction amount were studied. This helps to understand the normal transaction range and identify unusually large or small transactions.

Transaction amount can be an important feature for fraud detection. A transaction with an unusual amount compared to normal customer behaviour may be considered suspicious. However, a high transaction amount alone does not mean that the transaction is fraudulent. It needs to be analysed along with other transaction features.

The amount analysis therefore helps in understanding the importance of transaction value while building the fraud detection model.

```
# Maximum Transaction
max_trans = df['amt'].max()
print('Maximum Transaction :',max_trans)

# Minimum Transaction
min_trans = df['amt'].min()
print('Minimum Transaction :',min_trans)

# Average Transaction
avg_trans = df['amt'].mean().round(2)
print('Average Transaction :',avg_trans)

# Median Transaction
med_trans = df['amt'].median()
print('Median Transaction :',med_trans)
```


11. DATA VISUALIZATION

10.1 Fraud vs Genuine Transactions

Purpose of the Chart:

This chart is used to compare the number of genuine and fraudulent transactions in the dataset. It helps to understand the distribution of the target variable `is_fraud`.

Chart Analysis:

The x-axis represents the `is_fraud` values. Here, **0 represents genuine transactions** and **1 represents fraudulent transactions**. The y-axis represents the number of transactions.

From the chart, there are approximately **553,574 genuine transactions** and **2,145 fraudulent transactions** out of a total of **555,719 transactions**.

Key Insights:

The chart clearly shows that genuine transactions are much higher than fraudulent transactions. Genuine transactions make up approximately **99.6%** of the dataset, while fraudulent transactions are only around **0.4%**.

This shows that the dataset is **highly imbalanced**. Because the number of fraudulent transactions is very small compared to genuine transactions, detecting the fraud class correctly is a major challenge for the Machine Learning model. Therefore, while evaluating the model, metrics such as **Precision, Recall, F1-score, and Confusion Matrix** are important along with accuracy.



10.2 Transaction Amount Distribution

Purpose of the Chart:

This histogram is used to understand how the transaction amounts are distributed in the dataset. It helps to identify the common transaction amount range and unusual or very high transaction values.

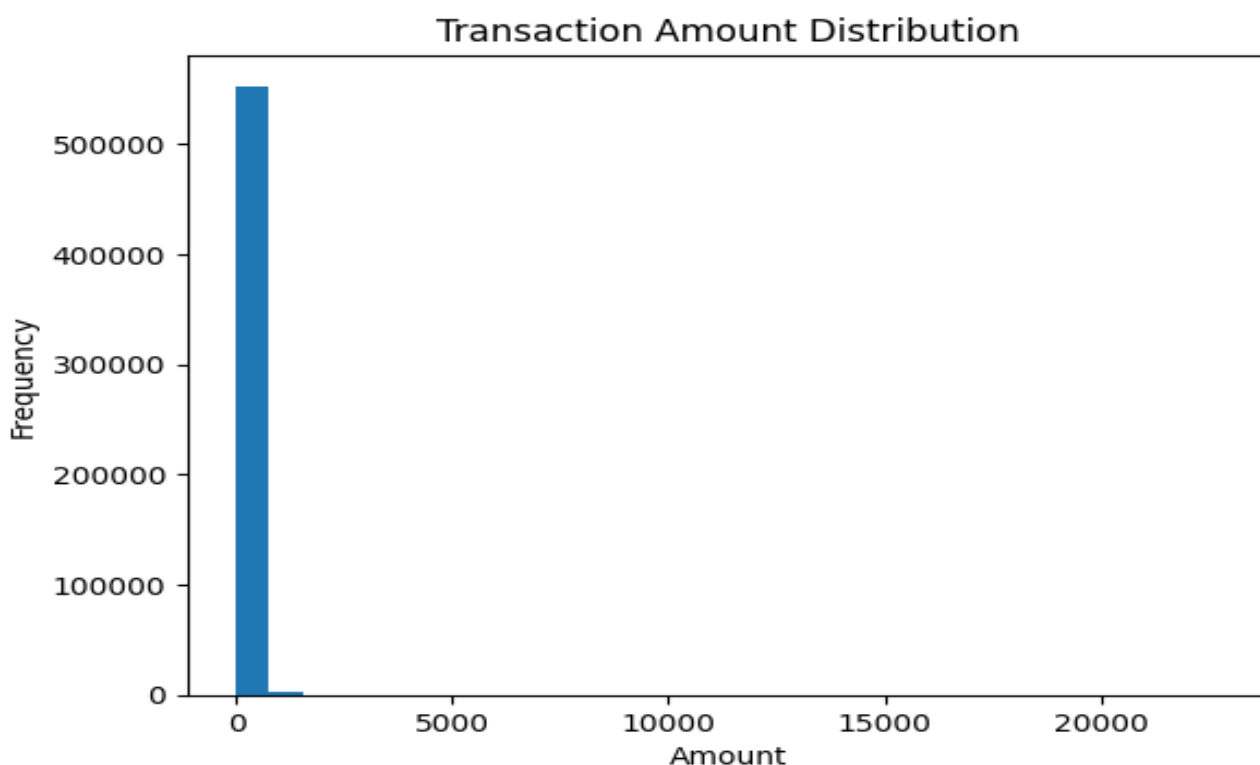
Chart Analysis:

The x-axis represents the **transaction amount**, while the y-axis represents the **frequency of transactions**. From the chart, most of the transactions are concentrated in the lower amount range. The highest frequency is seen near the lower transaction amounts, with the frequency reaching approximately **550,000 transactions**.

As the transaction amount increases, the number of transactions decreases significantly. The chart also shows that some transactions have very high amounts, reaching above **20,000**. However, these high-value transactions are very few compared to the transactions with lower amounts.

Key Insights:

The main observation is that most transactions in the dataset are for **smaller amounts**, while very high-value transactions occur rarely. The distribution is therefore **right-skewed**, with a long tail towards higher transaction amounts. The presence of very high transaction amounts also indicates that the amt feature contains **outlier values**. These values should be analysed carefully because an unusually large transaction amount can sometimes be associated with suspicious activity, although a high amount alone does not mean that the transaction is fraudulent.



10.3 Transaction Amount Box Plot

Purpose of the Chart:

This box plot is used to identify the spread of transaction amounts and detect unusual or extreme values in the amt column. It helps to understand whether the transaction amounts contain outliers.

Chart Analysis:

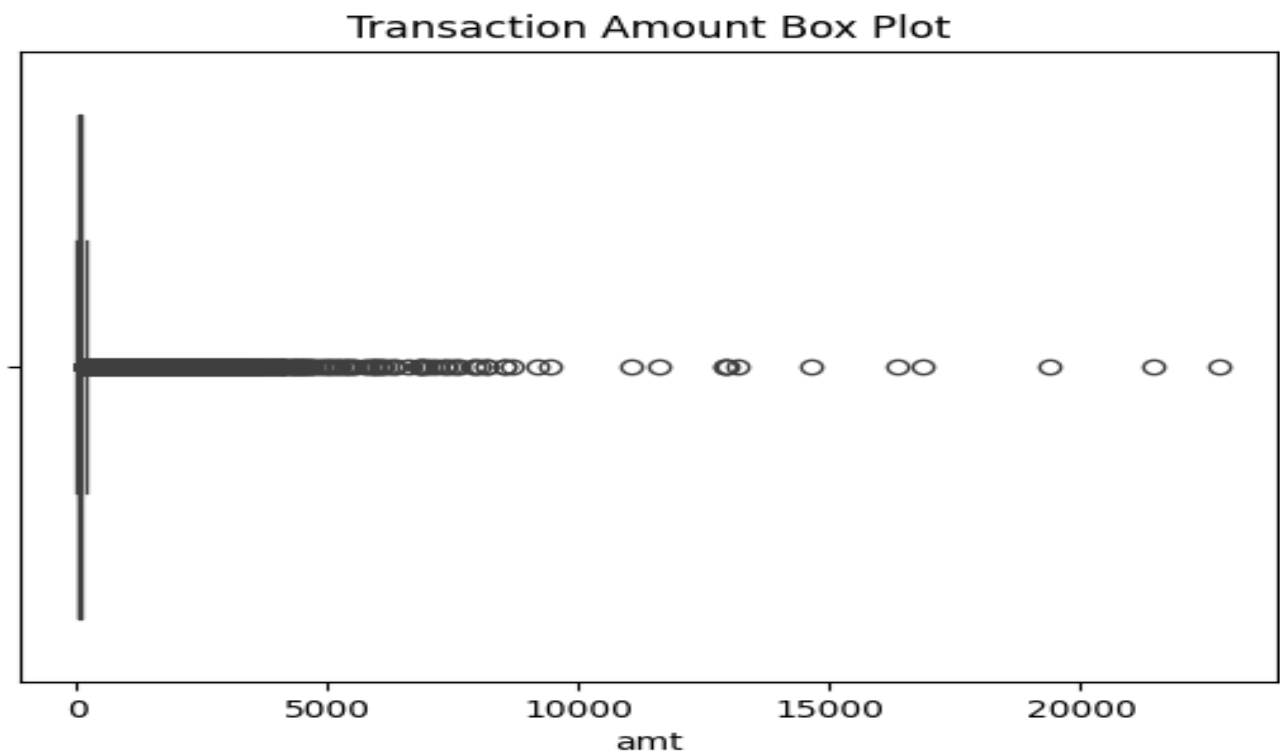
The x-axis represents the **transaction amount**. The box plot shows that most transaction amounts are concentrated near the lower range. The main box and whiskers are very close to the beginning of the x-axis, which indicates that most transactions have relatively small amounts.

Several individual points can be seen far away from the main box. These points represent **outliers** in the transaction amount. Some of the extreme values are above **20,000**, with the highest visible value reaching approximately **22,500**.

Key Insights:

The chart shows that the amt column contains a large number of **outlier values**. Most transactions have smaller amounts, while a small number of transactions have unusually high amounts.

These outliers are important for the fraud detection project because unusually high transaction amounts may indicate suspicious behaviour. However, they should not be removed directly because a high-value transaction can also be a genuine transaction. Therefore, the outliers need to be analysed carefully before deciding how to handle them.



10.4 Fraud Distribution

Purpose of the Chart:

This pie chart is used to show the percentage of genuine and fraudulent transactions in the dataset. It gives a simple view of how the two transaction classes are distributed.

Chart Analysis:

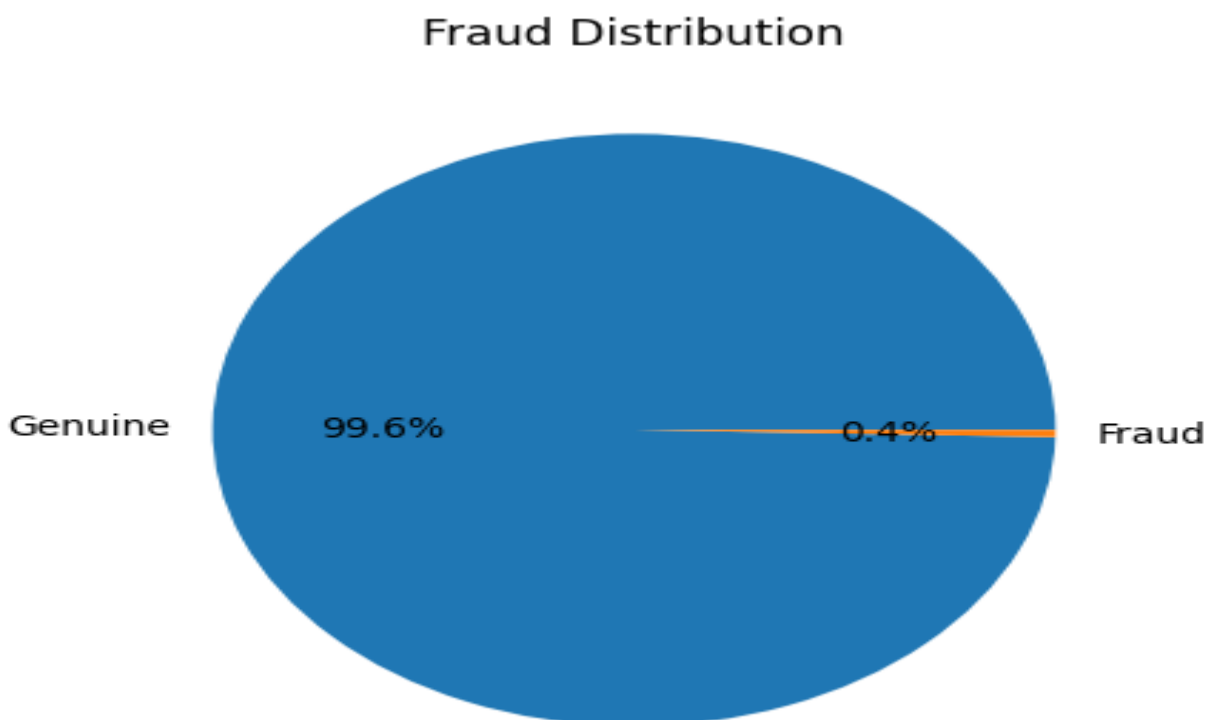
The chart shows two categories: **Genuine** and **Fraud**. Genuine transactions represent **99.6%** of the total transactions, while fraudulent transactions represent only **0.4%**.

Based on the dataset, there are approximately **553,574 genuine transactions** and **2,145 fraudulent transactions** out of **555,719 total transactions**.

Key Insights:

The chart clearly shows that genuine transactions are much higher than fraudulent transactions. Only a very small portion of the total transactions are fraudulent.

This confirms that the dataset is **highly imbalanced**. Because fraudulent transactions form only **0.4%** of the dataset, the Machine Learning model needs to identify this small class correctly. This is why metrics such as **Precision, Recall, F1-score, and Confusion Matrix** are important when evaluating the fraud detection model.



10.5 Transaction Amount by Fraud Status

Purpose of the Chart:

This box plot is used to compare the transaction amounts of **genuine and fraudulent transactions**. It helps to understand whether there is any difference in the transaction amount between the two classes and also helps to identify outliers.

Chart Analysis:

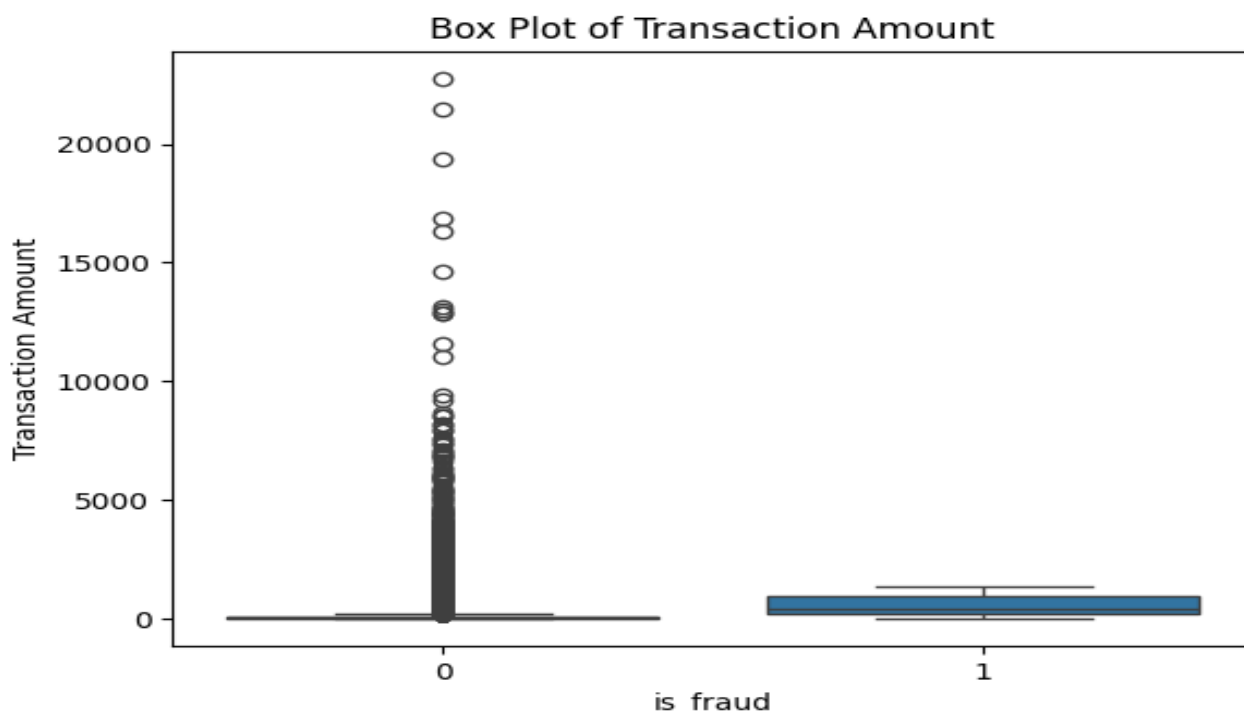
The x-axis represents the `is_fraud` variable, where **0 represents genuine transactions** and **1 represents fraudulent transactions**. The y-axis represents the transaction amount.

The box plot shows that genuine transactions have a much wider range of values and contain many outliers. Some genuine transactions have very high amounts, reaching above **20,000**. On the other hand, fraudulent transactions are concentrated within a comparatively smaller range, with their transaction amounts mostly staying below around **1,500**.

Key Insights:

The chart shows that the transaction amount patterns are different for genuine and fraudulent transactions. Genuine transactions have a large number of extreme values, while fraudulent transactions are more concentrated within a smaller amount range.

This indicates that **transaction amount can be a useful feature for fraud detection**. However, the presence of high-value genuine transactions also shows that transaction amount alone cannot be used to decide whether a transaction is fraudulent. It should be considered along with other features in the Machine Learning model.



10.6 Correlation Heatmap

Purpose of the Chart:

This heatmap is used to understand the relationship between different numerical features in the dataset. It shows the correlation between two variables using values ranging from **-1 to +1**. A value close to **+1** indicates a strong positive relationship, while a value close to **-1** indicates a strong negative relationship. A value near **0** indicates little or no linear relationship.

Chart Analysis:

The heatmap shows the correlation between features such as cc_num, amt, zip, lat, long, city_pop, unix_time, merch_lat, merch_long, is_fraud, and hour.

Some strong relationships can be seen between the location-related features. For example, long and merch_long have a correlation of approximately **1.00**, while lat and merch_lat have a correlation of approximately **0.99**. The zip feature also has a strong negative correlation with long and merch_long, both around **-0.91**.

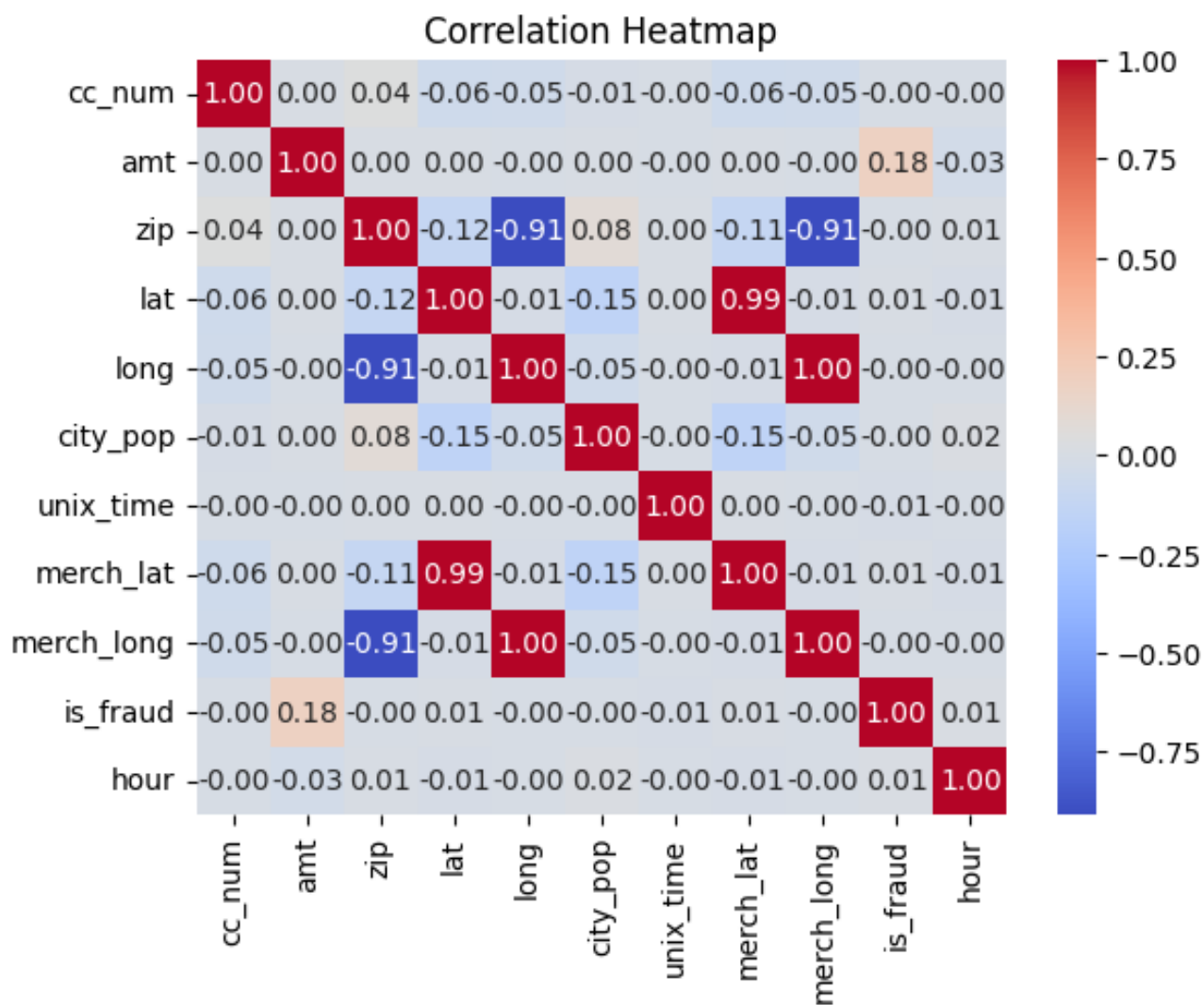
The correlation between amt and is_fraud is approximately **0.18**, which is the strongest visible relationship between the target variable and the numerical features in this heatmap. This indicates a **weak positive relationship** between transaction amount and fraud status.

Most of the other features have correlation values close to **0** with is_fraud, which means they do not show a strong linear relationship with fraud on their own.

Key Insights:

- amt has a correlation of approximately **0.18** with is_fraud, showing a weak positive relationship.
- lat and merch_lat have a very strong positive correlation of about **0.99**.
- long and merch_long have an almost perfect positive correlation of **1.00**.
- zip has a strong negative correlation of around **-0.91** with both long and merch_long.
- Most numerical features have a weak or almost zero correlation with is_fraud.
- The heatmap also shows that some location features are highly correlated with each other, which may be important when selecting features for the Machine Learning model.

Important: Correlation shows a statistical relationship between variables; it does **not** mean that one variable directly causes another.



12. FEATURE SCALING

11.1 Introduction to Feature Scaling

Feature scaling is an important data preprocessing step used before training a Machine Learning model. In a dataset, different features can have different ranges of values. For example, in the credit card transaction dataset, the transaction amount, city population, latitude, and longitude have different value ranges.

If these features are directly given to a Machine Learning algorithm, features with larger values may have more influence than features with smaller values. Therefore, feature scaling is used to bring numerical features into a similar range. In this project, feature scaling was performed on the selected numerical features before training the Machine Learning model. This helped to prepare the data in a suitable format for the model.

11.2 Need for Feature Scaling

The credit card transaction dataset contains numerical features with different scales. For example, the amt column contains transaction amounts that can have values ranging from small amounts to very large amounts. Similarly, city_pop can contain large population values, while features such as lat and long contain comparatively smaller values.

Because of these differences, the Machine Learning model may give more importance to features having larger numerical values. Feature scaling helps to reduce this problem by bringing the selected features to a common scale. Scaling is especially useful for algorithms that are affected by the magnitude of features. It can help the model learn more effectively and can also improve the overall performance of the model.

11.3 Scaling Method Used

In this project, **StandardScaler** is used for feature scaling. StandardScaler is available in the sklearn.preprocessing module.

StandardScaler changes the values of a feature so that they are centred around zero with a standard deviation of one. It uses the mean and standard deviation of each feature for the transformation.

The basic formula used by StandardScaler is:

$$z = (x - \mu) / \sigma$$

Where:

- x = original value
- μ = mean of the feature
- σ = standard deviation of the feature
- z = scaled value

After scaling, the values are not necessarily between 0 and 1. Instead, most values are generally distributed around zero.

11.4 Features Selected for Scaling

The numerical features that are useful for the Machine Learning model are considered for scaling. In the credit card fraud dataset, numerical columns include features such as amt, lat, long, city_pop, merch_lat, merch_long, and other numerical features created during preprocessing.

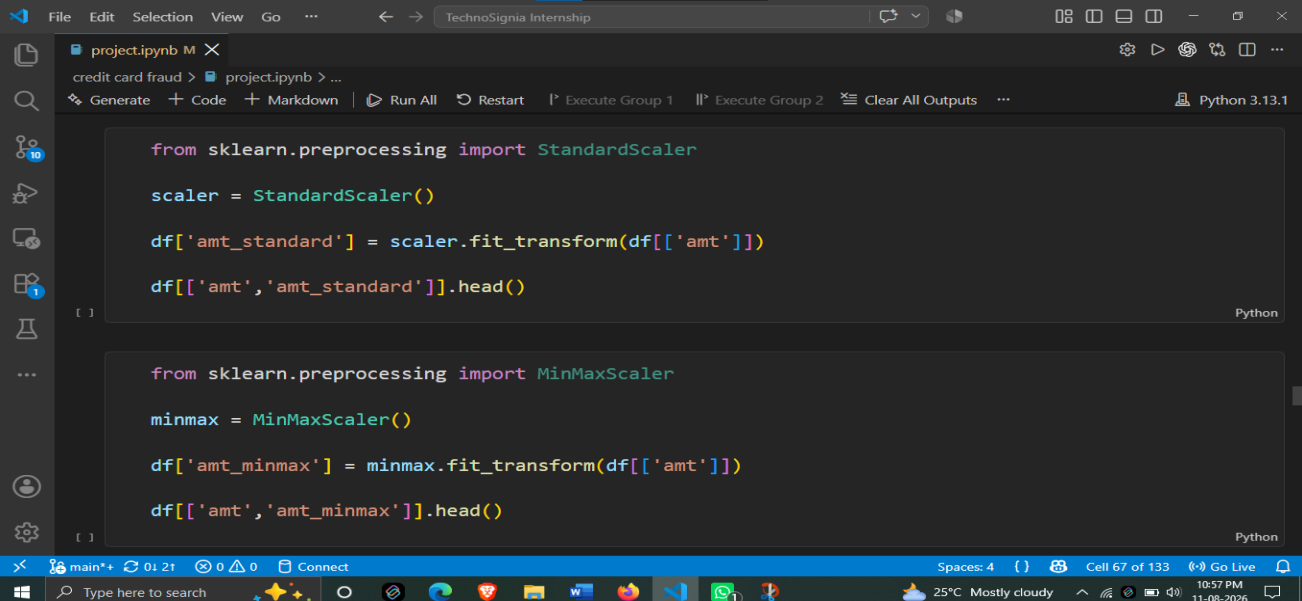
Categorical information such as merchant, category, gender, city, and job is not directly scaled because these columns contain categorical values. They are first converted into a suitable numerical form if they are required for model training.

The target variable is_fraud is also not scaled because it represents the output class of the model, where **0 represents genuine transactions and 1 represents fraudulent transactions**.

11.5 Implementation of Feature Scaling

Feature scaling is implemented using the StandardScaler class from Scikit-learn. First, the scaler is created and then it is fitted on the training data. After fitting, the selected numerical features are transformed into their scaled values.

A basic implementation is:



```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df['amt_standard'] = scaler.fit_transform(df[['amt']])

df[['amt', 'amt_standard']].head()

from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()

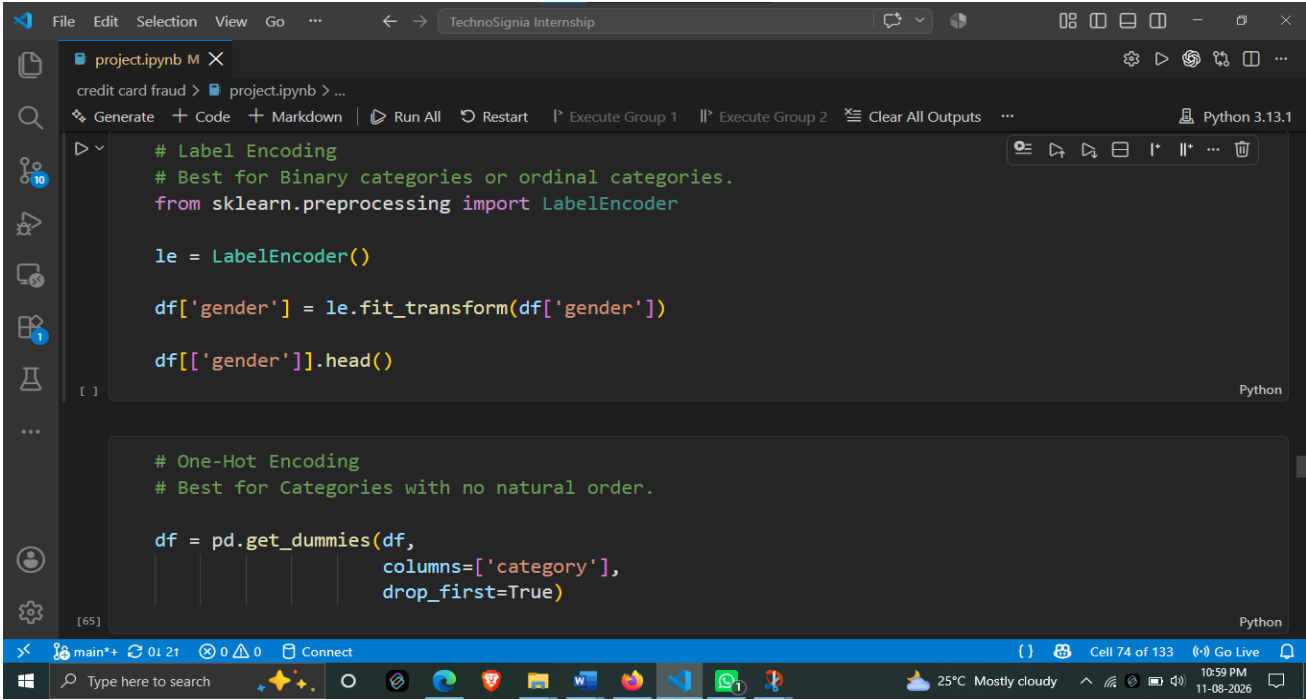
df['amt_minmax'] = minmax.fit_transform(df[['amt']])

df[['amt', 'amt_minmax']].head()
```

Here, StandardScaler() creates the scaling object. The fit_transform() method is used on the training data to calculate the mean and standard deviation and then transform the training features.

For the testing data, only the transform() method is used. This is important because the testing data should be transformed using the same scaling information learned from the training data.

This process helps to avoid data leakage from the testing dataset into the training process.

A screenshot of a Jupyter Notebook interface. The top bar shows 'TechnoSignia Internship' and various window controls. The notebook is titled 'project.ipynb'. The code is written in Python 3.13.1. The first section is for Label Encoding, showing the import of LabelEncoder from sklearn.preprocessing, its instantiation, and its use to transform the 'gender' column of a dataframe 'df'. The second section is for One-Hot Encoding, showing the use of pd.get_dummies to create dummy variables for the 'category' column, with 'drop_first=True' to avoid multicollinearity. The interface includes a left sidebar with icons for file explorer, search, and other Jupyter features. The bottom status bar shows 'main+', '01.21', '0 0', 'Connect', 'Cell 74 of 133', 'Go Live', and system information like '25°C Mostly cloudy' and '10:59 PM 11-08-2026'.

11.6 Before and After Feature Scaling

Before scaling, the numerical features in the dataset have different ranges. For example, transaction amount and city population can contain much larger values compared to latitude and longitude. Because of this difference, the features are not on the same scale.

After applying StandardScaler, the selected numerical features are transformed to a common scale. The values are centred around zero, and the effect of different feature ranges is reduced.

For example, a feature with very large numerical values will no longer dominate a feature with smaller values only because of its scale. This allows the Machine Learning model to consider the selected features more fairly.

Feature scaling therefore prepares the numerical data for the next stage of the project, where the scaled features are used for **Machine Learning model training and evaluation**.

13. TRAIN-TEST SPLIT

12.1 Introduction to Train-Test Split

Train-Test Split is an important step in Machine Learning. After preprocessing the dataset, the available data is divided into two parts: **training data** and **testing data**.

The training data is used to teach the Machine Learning model. The model learns the patterns and relationships between the input features and the target variable from this data. The testing data is kept separate and is used after training to check how well the model performs on new and unseen data.

In this project, the credit card transaction dataset is divided into training and testing sets before building the fraud detection model.

12.2 Why Train-Test Split is Used

The main purpose of Train-Test Split is to check whether the Machine Learning model can perform well on data that it has not seen during training.

If the model is trained and tested on the same data, it may simply learn the training data very well and give a high accuracy. However, this does not mean that the model will work well on new transactions. This problem is known as **overfitting**.

By keeping a separate testing dataset, we can check the actual performance of the model on unseen data. This gives a better idea of whether the model has learned useful patterns or has simply memorised the training data.

12.3 Importance in Machine Learning

Train-Test Split is important because it helps to evaluate the **generalization ability** of a Machine Learning model. Generalization means how well a trained model can make predictions on new data.

For a Credit Card Fraud Detection system, this is very important. In real life, the model will receive new transactions that were not present in the training dataset. Therefore, the model should be able to correctly identify fraudulent and genuine transactions in unseen data.

The testing data provides an independent way to measure the model's performance. Metrics such as **Accuracy, Precision, Recall, F1-score, and Confusion Matrix** can then be calculated using the predictions made on the test data.

12.4 Implementation

The `train_test_split()` function from Scikit-learn can be used to divide the dataset:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Here:

- X represents the input features.
- y represents the target variable, which is `is_fraud`.
- X_train contains the features used for training.
- X_test contains the features used for testing.
- y_train contains the target values for training.
- y_test contains the target values for testing.
- `test_size=0.2` means **20% of the data is used for testing and 80% for training**.
- `random_state=42` ensures that the same split can be reproduced whenever the code is run again.
-

12.5 Train and Test Data in This Project

In this project, **80% of the available data is used for training the Machine Learning model and 20% is used for testing the model**.

The model learns the relationship between transaction features and the `is_fraud` target from the training data. After training, the test data is given to the model to check whether it can correctly classify transactions as genuine or fraudulent. Since this project deals with an imbalanced dataset, maintaining a proper train-test split is particularly important. It helps ensure that the model is evaluated on data that was not used during training and gives a more realistic measure of its fraud detection performance.

14. DATA IMBALANCE HANDLING

13.1 Introduction to Data Imbalance

Data imbalance is a common problem in classification-based Machine Learning projects. It occurs when the number of records in one class is much higher than the number of records in another class.

In this Credit Card Fraud Detection project, the target variable is `is_fraud`. The value **0 represents a genuine transaction**, while **1 represents a fraudulent transaction**. From the dataset analysis, there are approximately **553,574 genuine transactions** and **2,145 fraudulent transactions** out of **555,719 total transactions**.

This means that genuine transactions are around **99.6%**, while fraudulent transactions are only around **0.4%**. Therefore, the dataset is highly imbalanced.

13.2 Need for Handling Data Imbalance

Handling data imbalance is important because a Machine Learning model can become biased towards the majority class. In this project, the majority class is the genuine transaction class.

For example, if a model predicts almost every transaction as genuine, it can still get very high accuracy because genuine transactions make up most of the dataset. However, such a model would not be useful for fraud detection because the main purpose of the project is to correctly identify fraudulent transactions. Therefore, the imbalance needs to be considered while preparing the data and training the model. The main focus is to make sure that the Machine Learning model gets enough information about the minority fraud class.

13.3 Class Distribution Before Handling Imbalance

Before handling the imbalance, the distribution of the target variable was analysed.

Transaction Type	<code>is_fraud</code> Value	Approximate Count	Percentage
Genuine	0	553,574	99.6%
Fraudulent	1	2,145	0.4%
Total	—	555,719	100%

The table clearly shows a large difference between the two classes. There are more than five lakh genuine transactions, while the number of fraudulent transactions is only a few thousand.

This imbalance makes fraud detection more challenging because the model has much more information about genuine transactions than fraudulent ones.

13.4 Data Imbalance Handling

To handle the imbalance, the training data needs to be prepared in a way that gives proper importance to both classes. Different techniques can be used for this purpose, such as **oversampling, undersampling, SMOTE, or class weighting**.

The choice of technique depends on the Machine Learning algorithm, dataset size, and project requirements. The main purpose of using any imbalance-handling technique is to improve the model's ability to identify the minority class without unnecessarily affecting the genuine transactions.

In this project, imbalance handling is performed after separating the data into training and testing sets. This is an important step because the test data should remain separate and should represent the original data distribution as much as possible.

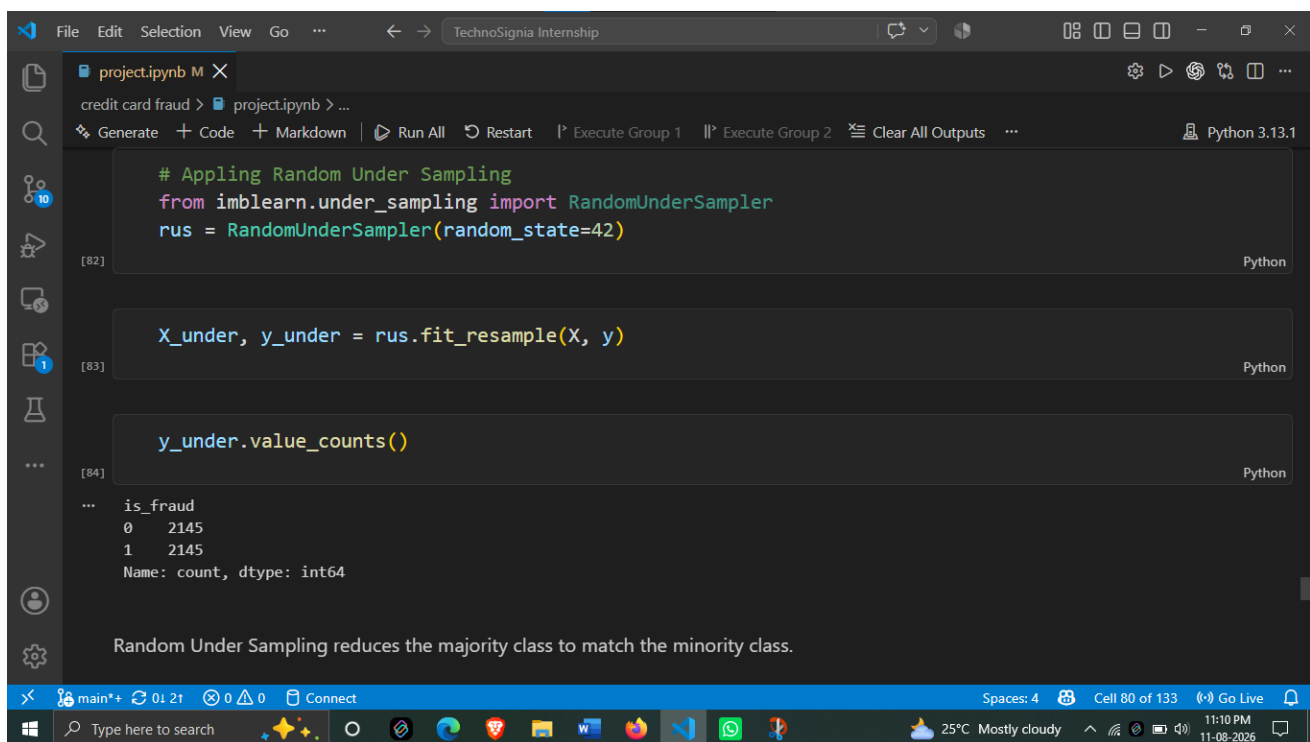
13.5 Implementation

The imbalance-handling process is applied to the training data before training the Machine Learning model. The features and target variable are first separated, and the training data is then prepared for handling the difference between genuine and fraudulent transactions.

The basic flow followed in the project is:

Dataset → Train-Test Split → Imbalance Handling → Model Training → Model Evaluation

This process allows the model to learn from a better representation of the fraud class while keeping the testing data separate for final evaluation.



The screenshot shows a Jupyter Notebook window titled 'TechnoSignia Internship'. The notebook contains three code cells. The first cell imports 'RandomUnderSampler' from 'imblearn.under_sampling' and creates an instance 'rus' with 'random_state=42'. The second cell uses 'rus.fit_resample(X, y)' to resample the data. The third cell prints 'y_under.value_counts()', which shows the following output:

```
is_fraud
0      2145
1      2145
Name: count, dtype: int64
```

Below the output, a text box states: 'Random Under Sampling reduces the majority class to match the minority class.'

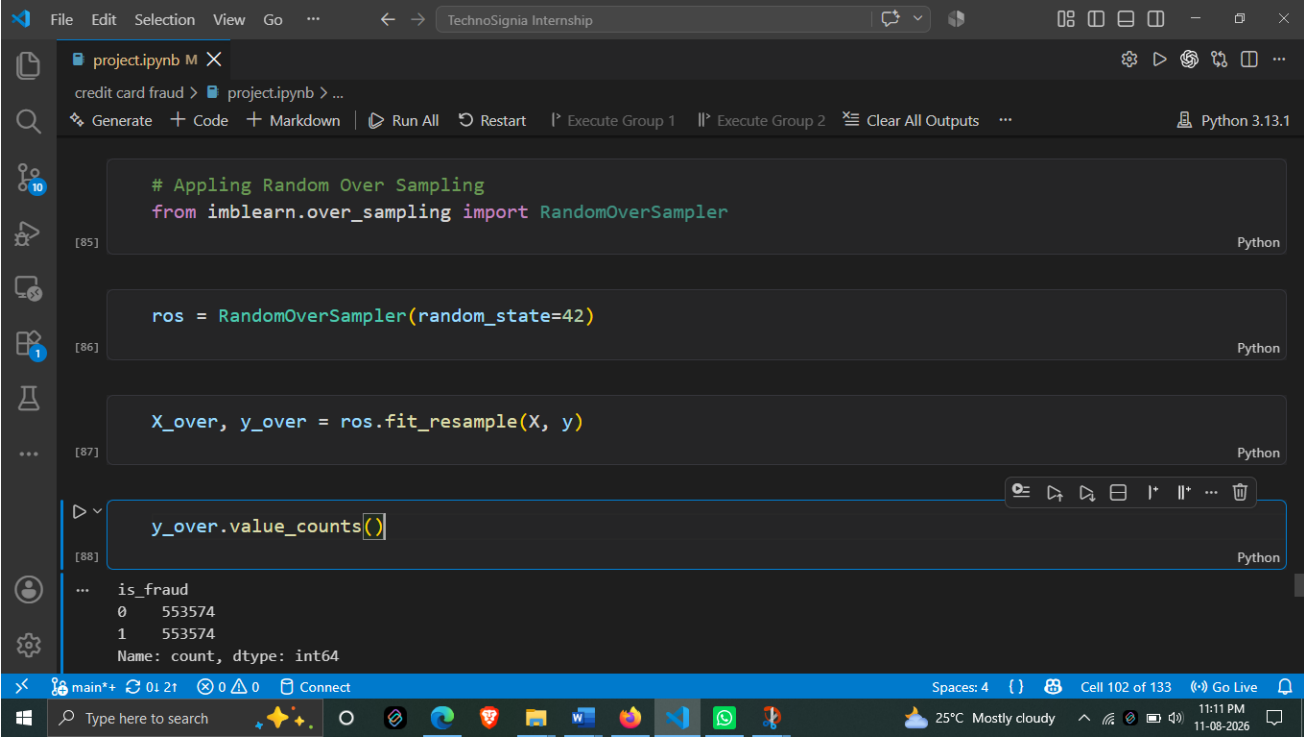
The balanced or adjusted training data is then given to the Machine Learning algorithm. The model learns patterns from both genuine and fraudulent transactions and uses these patterns to make predictions.

13.6 Class Distribution After Handling Imbalance

After applying the selected imbalance-handling method, the class distribution of the training data is checked again. This step is performed to confirm whether the imbalance has been reduced and whether the minority fraud class has received sufficient representation.

The before-and-after class distribution is important because it helps to verify whether the applied technique has worked as expected.

The testing dataset is not used for balancing because it should be kept separate from the training process. This helps in getting a more realistic evaluation of how the trained model performs on unseen transaction data.



The screenshot shows a Jupyter Notebook window titled 'TechnoSignia Internship'. The notebook contains the following code cells:

```
[85] # Applying Random Over Sampling
from imblearn.over_sampling import RandomOverSampler

[86] ros = RandomOverSampler(random_state=42)

[87] X_over, y_over = ros.fit_resample(X, y)

[88] y_over.value_counts()
```

The output of the last cell is displayed as follows:

```
is_fraud
0    553574
1    553574
Name: count, dtype: int64
```

The bottom status bar of the Jupyter Notebook shows 'main*+', '0/1 2/1', '0/0 0/0', 'Connect', 'Spaces: 4', 'Cell 102 of 133', 'Go Live', and the system clock '11:11 PM 11-08-2026'.

13.7 Importance of Data Imbalance Handling in Fraud Detection

Data imbalance handling is particularly important in credit card fraud detection because fraudulent transactions are much less common than genuine transactions. If the imbalance is ignored, the model may become more focused on predicting genuine transactions and may fail to identify some fraudulent transactions.

By giving the fraud class proper importance during training, the model can learn more useful patterns related to fraudulent transactions. This can help improve the model's ability to detect fraud.

15. MODEL BUILDING

After completing data preprocessing, feature scaling, train-test split, and data imbalance handling, the next step is to build Machine Learning models. Since the target variable is `is_fraud` contains two classes, **0 for genuine transactions and 1 for fraudulent transactions**, this is a binary classification problem.

In this project, four different Machine Learning algorithms are used and compared:

1. Logistic Regression
2. Decision Tree Classifier
3. Random Forest Classifier
4. K-Nearest Neighbors (KNN)

Using different models helps to understand which algorithm performs better for the credit card fraud detection problem.

14.1 Logistic Regression

Logistic Regression is a classification algorithm used to predict the probability of a particular class. In this project, it is used to classify a transaction as either genuine or fraudulent.

The model takes the input transaction features and calculates the probability of the transaction belonging to the fraud class. Based on the probability, the transaction is classified into one of the two classes.

Logistic Regression is simple and easy to understand. It is also useful as a baseline model for comparing the performance of other classification algorithms.

Implementation

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(max_iter=1000)
```

```
lr_model.fit(X_train, y_train)
```

```
y_pred_lr = lr_model.predict(X_test)
```

Here, `LogisticRegression()` creates the model. The `max_iter=1000` parameter gives the model enough iterations to find a suitable solution during training. `fit()` trains the model using the training data, while `predict()` is used to make predictions on the test data.

14.2 Decision Tree Classifier

Decision Tree is a supervised Machine Learning algorithm that makes predictions using a tree-like structure. It divides the data into different branches based on conditions applied to the features.

For example, the model may check whether a particular feature is greater or smaller than a certain value. Based on the answer, it moves to another branch until it reaches a final decision.

In this project, Decision Tree is used to classify transactions as genuine or fraudulent. One advantage of this algorithm is that its decision-making process is relatively easy to understand.

Implementation

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_model = DecisionTreeClassifier(random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

```
y_pred_dt = dt_model.predict(X_test)
```

Here, DecisionTreeClassifier() creates the Decision Tree model. The random_state=42 parameter is used to make the results reproducible. The model is trained using fit() and predictions are generated using predict().

14.3 Random Forest Classifier

Random Forest is an ensemble Machine Learning algorithm. Instead of using only one Decision Tree, it creates multiple Decision Trees and combines their predictions to make the final prediction.

This approach generally makes the model more stable and reduces the chances of depending too much on a single tree. Random Forest can also handle a large number of features and is useful for classification problems.

In this project, Random Forest is used to identify patterns in credit card transactions and classify them as genuine or fraudulent.

Implementation

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

```
rf_model.fit(X_train, y_train)
```

```
y_pred_rf = rf_model.predict(X_test)
```

Here, n_estimators=100 means that the Random Forest creates **100 decision trees**. The predictions from these trees are combined to produce the final result. random_state=42 is used to get consistent results when the model is run again.

14.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors, commonly known as KNN, is a classification algorithm that makes predictions based on the nearest data points.

When a new transaction is given to the model, KNN looks at the selected number of nearby transactions and checks their classes. The class that gets the majority of votes is used as the prediction for the new transaction.

In this project, KNN can be used to identify whether a transaction is more similar to genuine transactions or fraudulent transactions.

KNN is sensitive to the scale of the features because it calculates the distance between data points. Therefore, **feature scaling is particularly important when using KNN.**

Implementation

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
knn_model.fit(X_train, y_train)
```

```
y_pred_knn = knn_model.predict(X_test)
```

Here, `n_neighbors=5` means that the model considers the **5 nearest data points** when making a prediction. The model compares these neighbouring points and assigns the class based on the majority vote.

14.5 Model Training Process

All four models are trained using the prepared training dataset. The same training and testing data are used for each algorithm so that their performance can be compared fairly.

The general process followed in the project is:

Preprocessed Data → Train-Test Split → Model Training → Prediction → Model Evaluation

After training, each model generates predictions for the test data. These predictions are then compared with the actual `is_fraud` values to evaluate the performance of each algorithm.

The models are evaluated using **Accuracy, Precision, Recall, F1-score, and Confusion Matrix**. These metrics are especially important in this project because the dataset contains a much smaller number of fraudulent transactions compared to genuine transactions.

14.5 Model Performance Comparison

After handling the data imbalance, four different Machine Learning models were trained and tested on the credit card transaction data. The accuracy of each model was calculated to compare their performance.

Logistic Regression achieved an accuracy of **52.68%**. Compared to the other models, its performance was lower on the prepared dataset.

Decision Tree Classifier achieved an accuracy of **96.04%**. It performed much better than Logistic Regression and was able to classify most of the test transactions correctly.

Random Forest Classifier gave the highest accuracy of **96.97%** among the four models. This shows that the Random Forest model performed best based on accuracy.

K-Nearest Neighbors (KNN) achieved an accuracy of **80.30%**. Its performance was better than Logistic Regression but lower than Decision Tree and Random Forest.

14.6 Accuracy Comparison

Sr. No.	Machine Learning Model	Accuracy
1	Logistic Regression	52.68%
2	Decision Tree	96.04%
3	Random Forest	96.97%
4	K-Nearest Neighbors (KNN)	80.30%

From the comparison, **Random Forest Classifier achieved the highest accuracy of 96.97%**, followed by Decision Tree with 96.04%. KNN achieved 80.30%, while Logistic Regression achieved 52.68%.

16.CONCLUSION AND FUTURE BENEFITS

15.1 Conclusion

The Credit Card Fraud Detection project was developed to identify fraudulent transactions using Machine Learning techniques. The main purpose of the project was to study credit card transaction data and build a system that can classify transactions as genuine or fraudulent.

The project started with understanding and analysing the transaction dataset.

Different data preprocessing steps were performed to prepare the data for Machine Learning. Exploratory Data Analysis helped to understand the transaction patterns, transaction amounts, time-related information, missing values, duplicate records, and relationships between numerical features.

One of the main challenges found during the project was the imbalance between genuine and fraudulent transactions. The dataset contained approximately **99.6% genuine transactions and 0.4% fraudulent transactions**. This showed that fraud detection is a challenging classification problem and that proper data preparation is important before training the model.

Four Machine Learning algorithms were implemented and compared: **Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors (KNN)**.

After handling the data imbalance, Logistic Regression achieved **52.68% accuracy**, Decision Tree achieved **96.04%**, Random Forest achieved **96.97%**, and KNN achieved **80.30%** accuracy.

Based on accuracy, **Random Forest Classifier performed the best among the four models**, with an accuracy of **96.97%**. The comparison also showed that different Machine Learning algorithms can give different results on the same dataset.

Overall, this project provided practical knowledge of data preprocessing, exploratory data analysis, feature scaling, train-test splitting, data imbalance handling, Machine Learning model building, and model evaluation. The project also showed how Machine Learning can be applied to a real-world financial problem such as credit card fraud detection.

15.2 Future Benefits and Scope

The Credit Card Fraud Detection system has good potential for future development. The project can be improved further by using larger and more recent transaction datasets. More transaction features can also be added to help the model understand customer behaviour more accurately.

In the future, the system can be developed for **real-time fraud detection**, where every transaction can be checked immediately before it is approved. This can help banks and financial institutions take quick action when suspicious activity is detected.

The system can also be improved by testing advanced Machine Learning and Deep Learning techniques. Different models can be compared to improve fraud detection performance, especially for detecting the small number of fraudulent transactions.

Another possible improvement is to create a user-friendly application or dashboard where banks can monitor suspicious transactions, view fraud trends, and receive alerts. This can make the system more useful for practical business applications.

The project can also help reduce financial losses, improve transaction security, protect customers from unauthorized transactions, and increase trust in digital payment systems. With further development and real-time integration, the system can become a useful tool for banks and other financial institutions.

17. REFERENCES

1. **Python Documentation** – Python programming language and programming concepts.
<https://docs.python.org/3/>
2. **Pandas Documentation** – Data cleaning, data manipulation, and data analysis.
<https://pandas.pydata.org/docs/>
3. **NumPy Documentation** – Numerical computing and data processing.
<https://numpy.org/doc/stable/>
4. **Matplotlib Documentation** – Data visualization and plotting.
<https://matplotlib.org/stable/>
5. **Seaborn Documentation** – Statistical data visualization.
<https://seaborn.pydata.org/>
6. **Scikit-learn Documentation** – Machine Learning algorithms, preprocessing, model selection, and evaluation.
<https://scikit-learn.org/stable/>
7. **Kaggle** – Credit Card Fraud Detection datasets and learning resources.
<https://www.kaggle.com/>
8. **Research papers and online articles** related to Credit Card Fraud Detection using Machine Learning were also referred to understand fraud detection techniques, data imbalance, classification algorithms, and model evaluation.